Bapuji Educational Association (Regd.)

Bapuji Institute of Engineering and Technology

An Autonomous Institute Affiliated to Visvesvaraya Technological University, Belagavi, Karnataka.

Department of Artificial Intelligence & Machine Learning

Analysis and Design of Algorithms (ADA)

Module 3

Space and Time Trade-Offs

- Sorting by Counting – Comparison Counting Sort
- Input Enhancement in String Matching – Horspool's Algorithm

---

Prepared by:

Dr. Naveen Kumar K R

(Exam Revision Notes)

Sorting by Counting: Comparison Counting Sort

1. What is Comparison Counting Sort?

- **Definition:** Comparison Counting Sort is a sorting algorithm that works by counting, for each element of a list, the total number of elements smaller than it.
- **Purpose:** It is primarily used for small arrays or in educational contexts to demonstrate the "space-for-time trade-off" technique.
- **Difference from Counting Sort:** Unlike standard distribution (frequency-based) counting sort, this method uses **pairwise comparisons** to compute the exact final position of every element.
- **Time Complexity:** The algorithm has a time complexity of $O(n^2)$ because it compares every possible pair of elements in the array.

2. Key Idea Behind Comparison Counting Sort

- The algorithm counts how many elements in the array are *strictly smaller* than a specific element.
- This count directly gives the **index** (starting from 0) where the element should be placed in the final sorted array.
- **Handling Duplicates:** When two elements are equal (e.g., $A[i] = A[j]$), the standard implementation uses an `else` branch to increment `Count[i]`. This ensures that duplicates are placed in different, consecutive positions, maintaining the correctness of the final sorted array.

3. Algorithm (Pseudocode)

The following pseudocode is the standard version of the algorithm:

```
Algorithm ComparisonCountingSort(A[0..n-1])
// Sorts array by comparison counting
// Input: Array A[0..n-1] of orderable elements
// Output: Array S[0..n-1] of A's elements sorted in nondecreasing order

for i ← 0 to n-1 do
    Count[i] ← 0

for i ← 0 to n-2 do
    for j ← i+1 to n-1 do
        if A[i] < A[j]
            Count[j] ← Count[j] + 1
        else
            Count[i] ← Count[i] + 1

for i ← 0 to n-1 do
    S[Count[i]] ← A[i]

return S
```

Explanation of Loops:

1. **First Loop:** Initializes the `Count` array with zeros.
2. **Nested Loops:** Perform pairwise comparisons. If $A[i]$ is smaller, $A[j]$'s count increases; otherwise, $A[i]$'s count increases.
3. **Final Loop:** Uses the computed counts as indices to place each element from the original array A into its correct position in the sorted array S.

4. Tracing the Algorithm for

Input array: `A = [5, 2, 7, 2, 5]`, `n = 5`.

Step 1 – Initialize `Count` array:

`Count = [0, 0, 0, 0, 0]`

Step 2 – Pairwise comparisons (all pairs i < j):

There are $\frac{n(n-1)}{2} = \frac{5\times4}{2} = 10$ comparisons in total.

Trace table with colour-coded branches

i	j	A[i]	A[j]	Comparison Result	Increment	Count array after step
0	1	5	2	5 < 2 ? **No**	Count[0]++	**[1,0,0,0,0]**
0	2	5	7	5 < 7 ? **Yes**	Count[2]++	**[1,0,1,0,0]**
0	3	5	2	5 < 2 ? **No**	Count[0]++	**[2,0,1,0,0]**
0	4	5	5	5 < 5 ? **No (else)**	Count[0]++	**[3,0,1,0,0]**
1	2	2	7	2 < 7 ? **Yes**	Count[2]++	**[3,0,2,0,0]**
1	3	2	2	2 < 2 ? **No (else)**	Count[1]++	**[3,1,2,0,0]**
1	4	2	5	2 < 5 ? **Yes**	Count[4]++	**[3,1,2,0,1]**
2	3	7	2	7 < 2 ? **No**	Count[2]++	**[3,1,3,0,1]**
2	4	7	5	7 < 5 ? **No**	Count[2]++	**[3,1,4,0,1]**
3	4	2	5	2 < 5 ? **Yes**	Count[4]++	**[3,1,4,0,2]**

Final `Count` array: `[3, 1, 4, 0, 2]`

Note: For equal elements (e.g., A=5 vs A=5), the `else` branch executes, correctly incrementing `Count[i]`.

Step 3 – Place elements into `S` array:

i	A[i]	Count[i]	S[Count[i]] ← A[i]	S array after this step
0	5	3	S[3] = 5	[_, _, _, 5, _]
1	2	1	S[1] = 2	[_, 2, _, 5, _]
2	7	4	S[4] = 7	[_, 2, _, 5, 7]
3	2	0	S[0] = 2	[2, 2, _, 5, 7]
4	5	2	S[2] = 5	**[2, 2, 5, 5, 7]**

5. Final Result

- **The final `Count` array:** `[3, 1, 4, 0, 2]`
- **The final sorted array `S`:** `[2, 2, 5, 5, 7]`
- **Number of comparisons:** 10 comparisons.

6. Examination Answer Writing Format

For a 10-mark question, use this layout:

1. **Definition & Complexity (2 marks):** Define Comparison Counting Sort and state $O(n^2)$.
2. **Pseudocode (2 marks):** Write the algorithm exactly as shown in Section 3.
3. **Initialization (1 mark):** Show the `Count` array starting with all zeros.
4. **Comparison Table (Traces) (4 marks):** Present the 10 comparisons in a table format. **This is the most important part for scoring.**
5. **Final Sorted Array (1 mark):** Show the `S` array and the final result.

7. Common Mistakes

- **Missing Comparisons:** Students often miss some pairs; ensure you show exactly 10 comparisons for $n = 5$.
- **Equality Logic:** Forgetting that if $A[i] = A[j]$, you must increment `Count[i]`. If you increment both or neither, the sorted array will be wrong.
- **Indexing Error:** Using `Count[i]` as the value to put in the array instead of using it as the **index**.
- **Confusing Algorithms:** Do not confuse this with distribution counting sort, which uses frequencies of values rather than pairwise comparisons.

8. Quick Revision

- **Core Logic:** `Count[i]` = number of elements strictly smaller than `A[i]` (plus handling for duplicates).
- **Efficiency:** $O(n^2)$ time, $O(n)$ extra space.
- **Comparison Rule:** If `A[i] < A[j]` then `Count[j]++`, else `Count[i]++`.
- **For Input `[5,2,7,2,5]`:** Final `Count = [3,1,4,0,2]`, Final `S = [2,2,5,5,7]`.

Input Enhancement in String Matching: Horspool's Algorithm

1. What is Horspool's Algorithm?

- **Definition:** Horspool's algorithm is a simplified version of the Boyer-Moore string-matching algorithm.
- **Purpose:** It is used to find the first occurrence (or all occurrences) of a **pattern** string within a larger **text** string.
- **Why it is faster:** Brute-force matching shifts the pattern only one position at a time. Horspool's algorithm uses "input enhancement" — it preprocesses the pattern to create a **shift table** that allows the algorithm to skip many characters in the text, making it much faster on average.

2. Key Idea Behind Horspool's Algorithm

- **Right-to-Left Comparison:** Unlike brute force, Horspool's algorithm compares the characters of the pattern with the text from **right to left**, starting with the last character of the pattern.
- **The Shift Table:** Before searching, the algorithm looks at the pattern and calculates how far to shift for every possible character in the alphabet.
- **The Role of T[i]:** When a mismatch occurs (or even after a match), the algorithm looks at the character in the text that is currently aligned with the **last character** of the pattern (let's call this character c or T[i]). It then looks up c in the shift table to determine the distance of the next shift.

3. Pattern Analysis (Shift Table Construction)

3.1 Shift Table Formula

The shift value for any character c is determined by its position in the pattern P of length m:

- If c is **not** among the first m−1 characters of the pattern, the shift value t(c) = m.
- If c **does** appear in the first m−1 characters, the shift value t(c) = m − 1 − j, where j is the **rightmost** index of c in P[0..m−2].

3.2 ShiftTable Algorithm (Pseudocode)

```
Algorithm ShiftTable(P[0..m−1])
// Fills the shift table used by Horspool's and Boyer-Moore algorithms
// Input: Pattern P[0..m−1] and an alphabet of possible characters
// Output: Table[0..size−1] indexed by alphabet's characters and
//         filled with shift sizes computed by formula (7.1)
for i ← 0 to size−1 do Table[i] ← m
for j ← 0 to m−2 do Table[P[j]] ← m − 1 − j
return Table
```

Explanation:

1. The first loop sets every character's shift to the full pattern length m.
2. The second loop goes from left to right (index 0 to m−2).
3. It **overwrites** the value every time it sees the character again. This ensures that only the **rightmost occurrence** (before the very last character) determines the final shift value.

3.3 Step-by-Step Shift Table Computation

For pattern `BAOBAB`, the length m = 6. We examine the first m−1 characters (`BAOBA`).

Shift Table Construction Steps

Step	j (index)	P[j]	m−1−j	Table[P[j]] updated to	Table entries after this step
Initialise	–	–	–	–	All = 6
Loop 1	0	B	6−1−0 = 5	Table['B'] = 5	B : 5
Loop 2	1	A	6−1−1 = 4	Table['A'] = 4	B : 5, A : 4
Loop 3	2	O	6−1−2 = 3	Table['O'] = 3	B : 5, A : 4, O : 3
Loop 4	3	B	6−1−3 = 2	Table['B'] = 2	B : 2, A : 4, O : 3
Loop 5	4	A	6−1−4 = 1	Table['A'] = 1	B : 2, A : 1, O : 3

Final Shift Table for BAOBAB:

Final Shift Table

Character (c)	Rightmost index in P[0..4]	Formula m−1−j	Shift Value t(c)
B	3	6 − 1 − 3	**2**
A	4	6 − 1 − 4	**1**
O	2	6 − 1 − 2	**3**
All others	–	Default (m)	**6**

4. Solving the Given Question – Complete Execution Trace

4.1 HorspoolMatching Algorithm (Pseudocode)

```
Algorithm HorspoolMatching(P[0..m-1], T[0..n-1])
// Implements Horspool's algorithm for string matching
// Input: Pattern P[0..m-1] and text T[0..n-1]
// Output: The index of the left end of the first matching substring
//         or -1 if there are no matches
ShiftTable(P[0..m-1]) // generate Table of shifts
i ← m - 1 // position of the pattern's right end
while i ≤ n - 1 do
    k ← 0 // number of matched characters
    while k ≤ m - 1 and P[m - 1 - k] = T[i - k] do
        k ← k + 1
    if k = m
        return i - m + 1
    else
        i ← i + Table[T[i]]
return -1
```

- **i:** Tracks the position in the text aligned with the **end** of the pattern.
- **k:** Tracks how many characters match from **right to left**.
- **i ← i + Table[T[i]]:** When a mismatch occurs, the pattern shifts based on the text character aligned with its tail.

4.2 Trace Table and Alignment Diagrams

Text: `BESS_KNEW_ABOUT_BAOBABS` (Length n = 23)
Pattern: `BAOBAB` (Length m = 6)

Complete Execution Trace

Pass	i	T[i]	Comparisons (k steps)	Mismatch?	Shift by Table[T[i]]	New i	Match?
1	5	K	k=0: P[5]=B vs T[5]=K → ✗	Yes	Table['K'] = 6	11	–
2	11	B	k=0: B vs B ✓ k=1: A vs A ✓ k=2: B vs _ → ✗	Yes	Table['B'] = 2	13	–
3	13	U	k=0: B vs U → ✗	Yes	Table['U'] = 6	19	–
4	19	B	k=0: B vs B ✓ k=1: A vs O → ✗	Yes	Table['B'] = 2	21	–
5	21	B	k=0 to k=5 all match ✓	No	–	–	**At 16**

4.3 Alignment Diagrams for All Passes

Pass 1 — i = 5 (Mismatch at 'K')

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
B	E	S	S	_	**K**	N	E	W	_	A	B	O	U	T	_	B	A	O	B	A	B	S
B	A	O	B	A	**B ✗**																	

Shift by Table['K'] = 6

Pass 2 — i = 11 (Mismatch at '_')

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
B	E	S	S	_	K	N	E	W	**_**	**A**	**B**	O	U	T	_	B	A	O	B	A	B	S
						B	A	O	**B ✗**	**A ✓**	**B ✓**											

Shift by Table['B'] (T[i] = T[11]) = 2

Pass 3 — i = 13 (Mismatch at 'U')

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
B	E	S	S	_	K	N	E	W	_	A	B	O	**U**	T	_	B	A	O	B	A	B	S
								B	A	O	B	A	**B ✗**									

Shift by Table['U'] = 6

Pass 4 — i = 19 (Mismatch at 'O' vs 'A')

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
B	E	S	S	_	K	N	E	W	_	A	B	O	U	T	_	B	A	**O**	**B**	A	B	S
														B	A	O	B	**A ✗**	**B ✓**			

Shift by Table['B'] (T[i] = T[19]) = 2

Pass 5 — i = 21 (MATCH FOUND)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
B	E	S	S	_	K	N	E	W	_	A	B	O	U	T	_	**B**	**A**	**O**	**B**	**A**	**B**	S
																B ✓	**A ✓**	**O ✓**	**B ✓**	**A ✓**	**B ✓**	
All six characters match — Pattern found at index 16!																						

5. Final Result

Metric	Value
Total alignments (passes)	**5**
Total comparisons made	1 + 3 + 1 + 2 + 6 = **13 comparisons**
Position of match (index)	**16** (zero-based)
Matching substring	**BAOBAB**

6. Examination Answer Writing Format

For a 7-Mark Question, Structure Your Answer as Follows:

Section	Marks	What to Include
ShiftTable Calculation	2	State the formula t(c) = m−1−j; show the step-by-step computation table; present the final Shift Table
Algorithm Execution	4	Write the HorspoolMatching pseudocode briefly; draw the Trace Table showing i, T[i], and the shift for every pass; provide Alignment Diagrams for all 5 passes
Final Statement	1	"The pattern BAOBAB is found in the text at index 16."

7. Common Mistakes to Avoid

⚠️ Watch Out for These Errors:

- **Incorrect 'B' shift:** Many students think t(B) = 6 because B is the last character. Remember: t(c) is based on the **rightmost occurrence in the first m−1 characters**. Thus, t(B) = 2.
- **Wrong Shift Character:** Always shift using the character at T[i] (the text character aligned with the pattern's end), **not** the character that caused the mismatch.
- **Indexing Errors:** Ensure you use zero-based indexing (0, 1, 2...). The match starts at 16, not 15.
- **Missing the Underscore:** Don't forget that the space (_) is a character. Its shift value is usually m (6 in this case).

8. Quick Revision

⏱️ One-Minute Revision Summary

Aspect	Key Point
Comparison direction	Right to Left
Shift rule	Use T[i] (text character aligned with pattern's end) with Table
Shift formula	t(c) = m − 1 − j (rightmost j in P[0..m−2]); default = m
Time complexity	Average case O(n), Worst case O(nm)
This problem result	Match at index **16**, 5 alignments, 13 total comparisons